

Étude Numérique de l'Interpolation

Réalisé par : Claude Delcasse, Thomas De Guzman
Encadrant : M. M. Ersoy
Date : 10 avril 2025

Table des matières

1 Introduction

2 Principe de l'Interpolation de Lagrange

- 2.1 Interpolation de Newton
- 2.2 Erreur d'interpolation

3 Phénomène de Runge

- 3.1 Exemple où le phénomène de Runge se produit
- 3.2 Erreur d'interpolation cas des points équi-répartis
- 3.3 Points de Tchebychev
- 3.4 Exemples d'interpolation des points de Tchebychev

4 Interpolation par morceaux

- 4.1 Interpolation affine
- 4.2 Interpolation par fonctions splines

5 Exemple d'application d'une étude numérique d'interpolation

6 Conclusion

7 Bibliographie

8 Annexes

1 Introduction

A travers ce TP, nous allons essayer de mettre en évidence certains phénomènes liés à l'interpolation des fonctions.

En effet, en analyse numérique il arrive souvent qu'on ne connaisse pas une fonction de manière exacte sur l'ensemble du domaine d'étude. Dans ce cas, on essaie alors de reconstruire cette fonction à partir de quelques valeurs connues. Dans la vie courante, on peut vouloir estimer la valeur d'une fonction entre deux mesures. On peut prendre comme exemple un TP de chimie où si on connaît la concentration d'un produit à $t = 5$ secondes et à $t = 20$ secondes, on peut essayer d'estimer sa valeur à $t = 10$ secondes. C'est ce qu'on appelle un **problème d'interpolation**.

Pour se faire, on suppose que la fonction f est une fonction bien définie sur un intervalle $[a, b]$. On cherche alors à l'approximer avec un polynôme $p(x)$ en utilisant les valeurs connues de f (et éventuellement de sa dérivée f') en certains points $x_0, x_1, \dots, x_n$.

Par exemple, on veut que le polynôme vérifie :

$$\begin{aligned} p(x_0) &= f(x_0) \\ p(x_1) &= f(x_1) \\ &\vdots \\ p(x_n) &= f(x_n) \end{aligned}$$

et ainsi que :

$$p'(x_n) = f'(x_n)$$

Le but est de trouver un polynôme p qui se rapproche le plus possible de $f(x)$, c'est-à-dire de minimiser l'erreur $|f(x) - p(x)|$.

Cela dépend :

- Du degré du polynôme,
- Des points choisis $(x_0, x_1, \dots, x_n)$,
- De la méthode utilisée (avec ou sans les dérivées, par exemple).

On cherche donc un polynôme de la forme :

$$p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

C'est-à-dire qu'on veut trouver les coefficients $a_0, a_1, \dots, a_n$ tels que :

$$\begin{cases} p(x_0) = f(x_0) \\ p(x_1) = f(x_1) \\ \vdots \\ p(x_n) = f(x_n) \end{cases}$$

2 Principe de l'Interpolation de Lagrange

Prenons $n \in \mathbb{N}^*$ et considérons $x_1, \dots, x_n \in \mathbb{K}$ qui sont distincts deux à deux. Pour chaque i appartenant à l'ensemble $\{1, \dots, n\}$, le polynôme interpolateur de Lagrange associé aux $x_1, \dots, x_n$ est défini comme étant le polynôme de degré $n - 1$.

$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j} \quad (1)$$

Les polynômes interpolateurs possèdent la propriété fondamentale suivante :

$$L_i(x_k) = \begin{cases} 1 & \text{si } i = k, \\ 0 & \text{si } i \neq k. \end{cases}$$

Donc, les polynômes de base de Lagrange forment une base de $\mathbb{P}_{n-1}$. [4]

Cela permet de fabriquer des polynômes de degré $n - 1$ qui prennent une valeur donnée en chaque x_i . Précisément, on a le théorème suivant :

Théorème : Soit $n \in \mathbb{N}^*$ et soient $x_1, \dots, x_n \in \mathbb{K}$ distincts deux à deux. On note L_i le i -ème polynôme de Lagrange associé aux $x_1, \dots, x_n$. Alors

— Pour toute famille $(b_1, \dots, b_n)$ d'éléments de $\mathbb{K}$, le polynôme

$$Q = \sum_{i=1}^n b_i L_i \quad (2)$$

est l'unique polynôme de degré inférieur ou égal à $n - 1$ et vérifie $Q(x_i) = b_i$ pour tout $i \in \{1, \dots, n\}$.

— Pour tout polynôme $P \in \mathbb{P}_{n-1}[X]$, on a

$$P = \sum_{i=1}^n P(x_i) L_i. \quad (3)$$

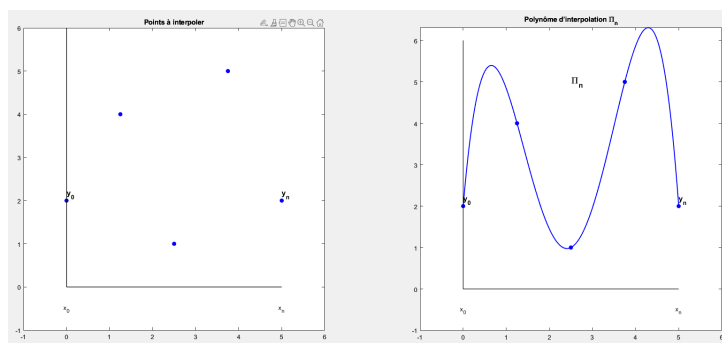


FIGURE 1 – Polynôme d'interpolation, n=4

Contrairement aux polynômes de Lagrange, l'interpolation de Newton, plus adaptée pour l'analyse numérique, construit le polynôme à l'aide des différences divisées, offrant ainsi une meilleure efficacité pour l'ajout de nouvelles données.

2.1 Interpolation de Newton

Les différences divisées permettent de discrétiser les dérivées successives d'une fonction. Soit $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ $(n+1)$ points distincts, les différences divisées successives sont définies par :

$$[y_k] = y_k, \quad k = 0, \dots, n$$

$$[y_k, \dots, y_{k+j}] = \frac{[y_{k+1}, \dots, y_{k+j}] - [y_k, \dots, y_{k+j-1}]}{x_{k+j} - x_k}, \quad k = 0, \dots, n-1, \quad j = 1, \dots, n-k.$$

Souvent, $y_j = f(x_j)$ pour une fonction f donnée, on note $f[x_0, \dots, x_n]$ la différence divisée $[y_0, \dots, y_n]$. [5]

La notion de différence divisée est utilisée pour exprimer le polynôme d'interpolation de Lagrange sous la forme de Newton :

$$L(x) = [y_0] + [y_0, y_1](x - x_0) + \dots + [y_0, \dots, y_n](x - x_0) \dots (x - x_{n-1}). \quad (4)$$

Le polynôme L est unique, de degré inférieur ou égal à n , et vérifie $L(x_k) = y_k$ pour tout $k = 0, \dots, n$.

2.2 Erreur d'interpolation

L'erreur d'interpolation lors de l'approximation d'une fonction f , avec $y_i = f(x_i)$, est donnée par une formule de type Taylor-Young :

Si f est $n+1$ fois différentiable sur $I = [\min(x_0, \dots, x_n, x), \max(x_0, \dots, x_n, x)]$, alors il existe $\xi \in I$ tel que

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i). \quad (5)$$

L'existence de ξ est démontrée par l'application itérée du théorème de Rolle.

Théorème de Rolle : Soient a et b deux réels tels que $a < b$, et f une fonction à valeurs réelles continue sur $[a, b]$ et dérivable sur (a, b) telle que $f(a) = f(b)$. Alors, il existe (au moins) un réel c dans (a, b) tel que $f'(c) = 0$.

En particulier, on a :

$$\|f - p_n\|_\infty \leq \frac{\|f^{(n+1)}\|_\infty}{(n+1)!} \sup_{x \in [a, b]} \left| \prod_{i=0}^n (x - x_i) \right| \quad (6)$$

où, $\|f\|_\infty = \sup_{x \in [a, b]} |f(x)|$.

Cependant, on n'a pas nécessairement :

$$\lim_{n \rightarrow \infty} \|f - p_n\|_\infty = 0.$$

Lorsque les points sont répartis de manière uniforme, c'est-à-dire $x_i = x_0 + ih$ pour tout $i \in \{1, \dots, n\}$ et $h \neq 0$, l'erreur d'interpolation peut devenir grande à mesure que le nombre de points augmente. Ce phénomène est connu sous le nom de phénomène de Runge.

3 Phénomène de Runge

Dans le cadre de l'interpolation polynomiale, notamment avec la méthode de Lagrange, augmenter le nombre de points d'interpolation ne garantit pas une meilleure approximation. Ce paradoxe, connu sous le nom de phénomène de Runge [6], est la conséquence de deux propriétés du problème :

- L'amplitude des dérivées de la fonction de Runge augmente très rapidement lorsque n augmente.
- L'équi-répartition des points d'interpolation mène à une constante de Lebesgue qui augmente très rapidement lorsque n augmente.

Les sous-sections suivantes illustrent ce phénomène et présentent une alternative efficace avec les points de Tchébychev.

3.1 Exemple où le phénomène de Runge se produit

Prenant les fonctions $f_1(x) = \cos(x)$ et $f_2(x) = \frac{1}{1+x^2}$ avec $x \in [-5, 5]$. Soit $n \in \mathbb{N}$, considérons la subdivision régulière suivante :

$$x_i = a + i \cdot \frac{b - a}{n}, \quad \text{avec } 1 \leq i \leq n.$$

On obtient alors les figures suivantes, générées avec MATLAB, représentant le tracé de la fonction exacte en bleu, et celui du polynôme d'interpolation en rouge.

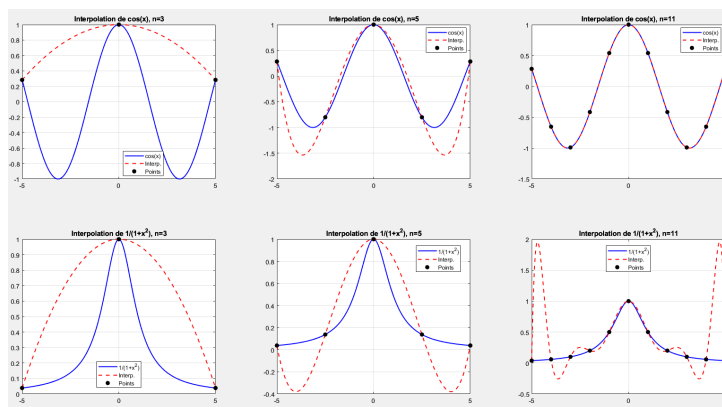


FIGURE 2 – Tracés avec l'interpolation de Lagrange

Pour la fonction f_1 , avec 3 points, le polynôme d'interpolation ne correspond à la fonction cosinus qu'en $x = 0, x = 5, x = -5$. Avec 5 et 11 points, l'interpolation est bien meilleure, mais dans le cas de 5 points, aux bords, le polynôme ne suit pas correctement la courbe.

Pour la fonction f_2 , on observe un phénomène similaire. Toutefois, même avec 11 points, le polynôme d'interpolation diverge (oscillations). Au bord, l'interpolation est mauvaise. De plus, en augmentant le nombre de points, le résultat se dégrade encore.

3.2 Erreur d'interpolation cas des points équi-répartis

Lorsque les points d'interpolation sont équidistants, la majoration du produit peut être effectuée. Dans ce cas,

$$\sup_{x \in [a,b]} \left| \prod_{i=0}^n (x - x_i) \right| \leq \frac{h^{n+1} n!}{4}$$

On obtient alors la majoration de l'erreur suivante,

$$|f - p_n|_{\infty} \leq \|f^{(n+1)}\|_{\infty} \frac{h^{n+1}}{4(n+1)} \quad (7)$$

Exemple numérique : Considérons la fonction $f(x) = 3^x$ sur l'intervalle $[-2, 2]$, interpolée en 5 points équidistants : $-2, -1, 0, 1, 2$. On cherche un polynôme $P(x)$ de degré 4 tel que $P(x_i) = f(x_i)$. Ce polynôme est obtenu en résolvant le système linéaire suivant :

$$A \cdot C = B$$

avec

$$A = \begin{bmatrix} 1 & -2 & 4 & -8 & 16 \\ 1 & -1 & 1 & -1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 & 16 \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{9} \\ \frac{1}{3} \\ 1 \\ 3 \\ 9 \end{bmatrix}$$

et C le vecteur des coefficients du polynôme $P(x)$. Une fois $P(x)$ calculé, on peut comparer numériquement $f(x)$ et $P(x)$ sur $[-2, 2]$. Le maximum de l'erreur $|f(x) - P(x)|$ sur cet intervalle est :

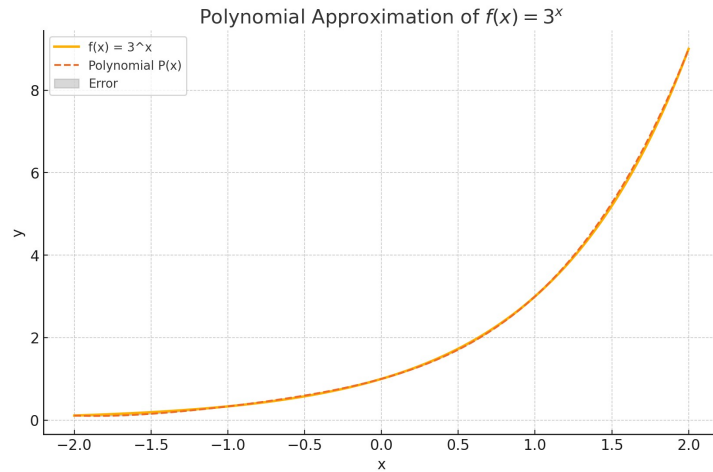


FIGURE 3 – Approximation polynomiale de $f(x) = 3^x$

$$\max_{x \in [-2,2]} |f(x) - P(x)| \approx 0,0775$$

Ce résultat illustre concrètement la qualité de l'interpolation polynomiale avec des points équidistants lorsque le nombre de points reste modéré.

3.3 Points de Tchebychev

Une solution consiste à ne pas choisir les points d'interpolation de façon uniforme dans l'intervalle $[-1, 1]$. L'une des raisons à cela repose sur l'inégalité suivante : si f est une fonction de classe $\mathcal{C}^N$ sur $[-1, 1]$, et si L désigne le polynôme d'interpolation de Lagrange aux points $x_1, \dots, x_N$, alors on a :

$$\forall x \in [-1, 1], \quad |f(x) - L(x)| \leq \sup_{x \in [-1, 1]} |(x - x_1)(x - x_2) \cdots (x - x_N)| \cdot \frac{\|f^{(N)}\|_{\infty}}{N!}$$

L'idée est donc de choisir les x_i de manière à minimiser :

$$\sup_{x \in [-1, 1]} |(x - x_1)(x - x_2) \cdots (x - x_N)|$$

Il est démontré que cette borne est minimale lorsque les x_i sont les racines du polynôme de Tchebychev de degré n , c'est-à-dire :

$$x_i = \cos\left(\frac{(2i - 1)\pi}{2n}\right), \quad i = 1, \dots, n$$

On peut généraliser cette méthode à n'importe quel intervalle fermé $[a, b]$, en appliquant une translation et une homothétie. Les abscisses de Tchebychev d'ordre n sur l'intervalle $[a, b]$ sont alors données par :

$$x_i = \frac{a + b}{2} + \frac{b - a}{2} \cos\left(\frac{(2i - 1)\pi}{2n}\right), \quad i = 1, \dots, n$$

Cette distribution permet de réduire l'erreur d'interpolation, notamment aux bords de l'intervalle, et d'éviter le phénomène de Runge observé avec des points équidistants.

3.4 Exemples d'interpolation des points de Tchebychev

On reprend les deux fonctions suivantes : $f_1(x) = \cos(x)$ et $f_2(x) = \frac{1}{1+x^2}$. On va refaire les calculs en utilisant la subdivision de Tchebychev.

Les points d'interpolation de Tchebychev d'ordre n sur un intervalle $[a, b]$ sont donnés par :

$$x_n = \frac{a + b}{2} + \frac{b - a}{2} \cos\left(\frac{(2 \cdot [0 : n - 1] + 1) \pi}{2 \cdot (n + 1)}\right)$$

On obtient alors les figures suivantes, générées avec MATLAB, représentant le tracé de la fonction exacte en bleu, et celui du polynôme d'interpolation en rouge.

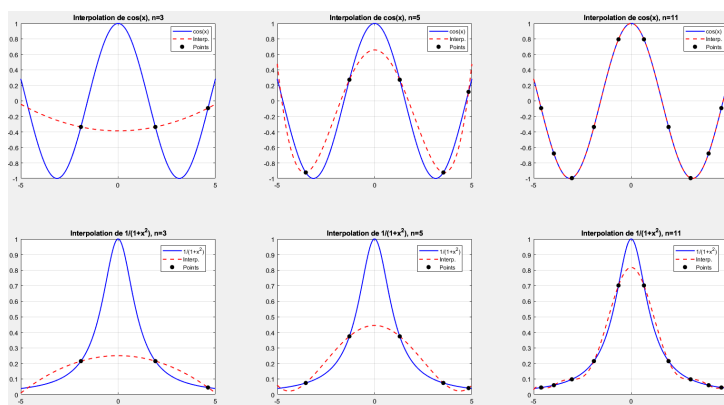


FIGURE 4 – Tracés avec l'interpolation de Tchebychev

4 Interpolation par morceaux

Après avoir utilisé l'interpolation polynomiale globale, si on se rend compte que l'on n'obtient pas de bons résultats, que ce soit à cause du **phénomène de Runge** ou de la complexité des polynômes, il est plus judicieux d'opter pour une approche locale, c'est-à-dire : *l'interpolation par morceaux*.

En effet, au lieu d'essayer de construire un seul polynôme sur tout l'intervalle $[a, b]$, avec le principe d'interpolation par morceaux consiste on découper cet intervalle en plusieurs petits sous-intervalles $[x_i, x_{i+1}]$, puis on construit, pour chacun d'eux, une fonction d'interpolation plus simple (souvent de faible degré) qui s'adapte localement aux données.

$$\forall x \in [x_i, x_{i+1}], \quad P(x) = P_i(x), \quad \text{où chaque } P_i \text{ est un polynôme adapté à l'intervalle.} \quad (8)$$

Cette approche offre plusieurs avantages : elle limite les oscillations et permet de mieux suivre les variations locales de la fonction. En effet, la modélisation épousera mieux la fonction étudiée sur chacun des sous-intervalles [1].

Dans les prochaines sections, nous allons étudier deux façons d'interpoler par morceaux en détaillant leur mise en œuvre, leurs points forts ainsi que leur limites :

- **L'interpolation affine**, qui consiste à relier chaque paire de points par une droite simple sur chaque sous-intervalles.
- **L'interpolation par fonctions splines**, qui utilise des polynômes plus lisses, généralement de degré 3, et qui assurent un meilleur lissage de l'approximation de la fonction sur chacun des sous-intervalles.

4.1 Interpolation affine

Définition. L'interpolation affine (ou linéaire) par morceaux consiste à approximer une fonction f continue sur un intervalle $[a, b]$ à l'aide de simples segments de droites reliant les points d'interpolation $(x_i, f(x_i))$ et $(x_{i+1}, f(x_{i+1}))$. Pour chaque sous-intervalle $[x_i, x_{i+1}]$, on définit un polynôme de degré 1 :

$$P_i(x) = f(x_i) + \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}(x - x_i), \quad \forall x \in [x_i, x_{i+1}]$$

L'ensemble des fonctions P_i définit une fonction $P(x)$ par morceaux qui est continue sur l'intervalle $[a, b]$.

Cette méthode offre plusieurs avantages. En effet, elle est plutôt simple à mettre en œuvre, elle offre une bonne approximation locale si la fonction n'est pas trop courbée et on n'a pas besoin de résoudre de système linéaire.

Cependant, la fonction résultante n'est pas dérivable aux points x_i (elle présente des discontinuités), en effet la fonction n'est pas $\mathcal{C}^1$ sur l'intervalle. De plus, on a une approximation qui est peu précise pour des fonctions qui varient beaucoup.

Théorème (Erreur d'interpolation affine). Soit $f \in \mathcal{C}^2([a, b])$ et $x \in [x_i, x_{i+1}]$. Il existe $\xi \in [x_i, x_{i+1}]$ tel que :

$$f(x) - P_i(x) = -\frac{(x - x_i)(x - x_{i+1})}{2} f''(\xi)$$

D'où une majoration uniforme de l'erreur :

$$|f(x) - P_i(x)| \leq \frac{(x_{i+1} - x_i)^2}{8} \cdot \max_{\xi \in [x_i, x_{i+1}]} |f''(\xi)|$$

Illustration graphique :

En appliquant la méthode d'interpolation par une fonction affine, on obtient les modélisations suivantes :

- Exemple 1 : Comparaison de l'interpolation de $f(x) = \frac{1}{(1+x^2)}$ sur $[-5, 5]$ pour 3, 5, et 11 points.

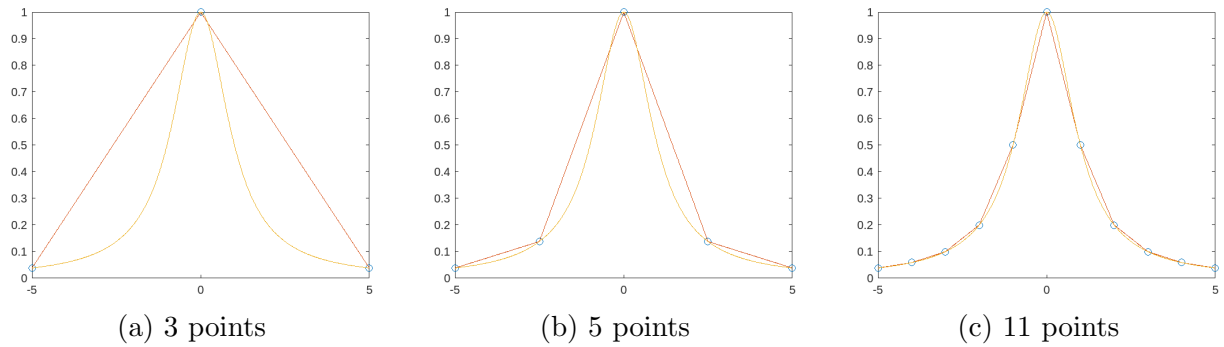


FIGURE 5 – Comparaison de l'interpolation de la fonction $f(x) = \frac{1}{(1+x^2)}$ sur $[-5, 5]$ pour 3, 5, et 11 points.

- Exemple 2 : Comparaison de l'interpolation de $f(x) = \cos(x)$ pour 3, 7 et 15 points.

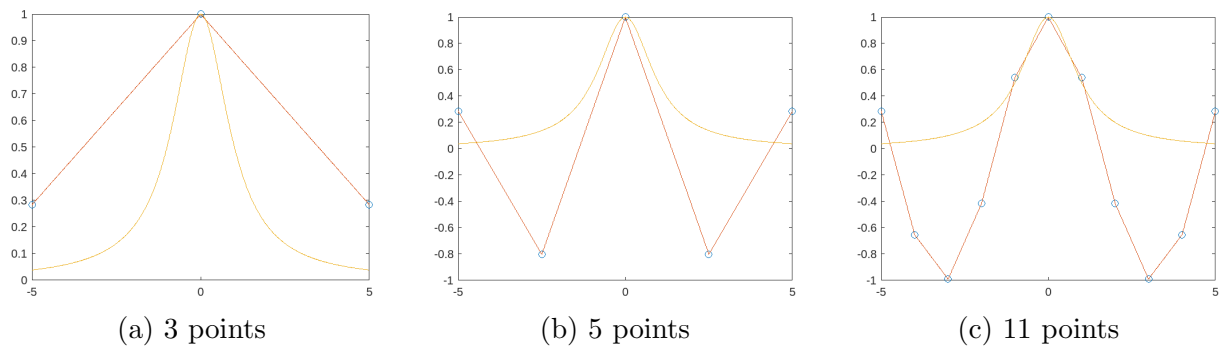


FIGURE 6 – Comparaison de l'interpolation de la fonction $f(x) = \cos(x)$ sur $[-5, 5]$ pour 3, 7 et 15 points.

4.2 Interpolation par fonctions splines

L'interpolation par une *fonction splines* est une méthode qui consiste à ajuster, sur chaque sous-intervalle $[x_i, x_{i+1}]$, un polynôme (souvent de degré 3) tout en assurant une régularité globale de la courbe approximant la fonction. C'est une alternative efficace à l'interpolation polynomiale globale, notamment lorsqu'on veut éviter les oscillations indésirables ou obtenir une courbe plus lisse. Le cas le plus courant est celui des **splines cubiques**.

Définition : Une spline cubique est une fonction $S(x)$, définie par morceaux sur chaque intervalle $[x_i, x_{i+1}]$, telle que :

- $S(x) \in \mathcal{C}^2([a, b])$, c'est-à-dire que S , S' et S'' sont continues.
- Sur chaque intervalle, $S|_{[x_i, x_{i+1}]}$ est un polynôme de degré 3.
- $S(x_i) = f(x_i)$, pour tout $i \in \{0, \dots, n\}$.

Pour déterminer les coefficients des polynômes cubiques, on impose des conditions supplémentaires :

- La continuité des dérivées premières et secondes aux nœuds x_i .
- Des conditions aux bords :
 - **Spline naturelle** : $S''(x_0) = S''(x_n) = 0$.
 - **Spline hermitienne** : on impose $S'(x_0) = f'(x_0)$ et $S'(x_n) = f'(x_n)$.

Cette méthode offre plusieurs avantages. En effet, elle offre une très bonne approximation des fonctions régulières. De plus, on observe qu'il n'y a pas d'oscillations exagérées comme on pourrait avoir avec des polynômes de degré élevés (on évite le phénomène de Runge). Et enfin, on obtient une fonction continue avec des dérivées lisses.

Cependant, cette fonction est plus complexe à implémenter et cette méthode nécessite la résolution de système linéaire ce qui la rend plus complexe.

Illustration graphique :

En appliquant la méthode d'interpolation par une fonction spline, on obtient les modélisations suivantes :

- Exemple 1 : Comparaison de l'interpolation de $f(x) = \frac{1}{(1+x^2)}$ sur $[-5, 5]$ pour 3, 5, et 11 points.

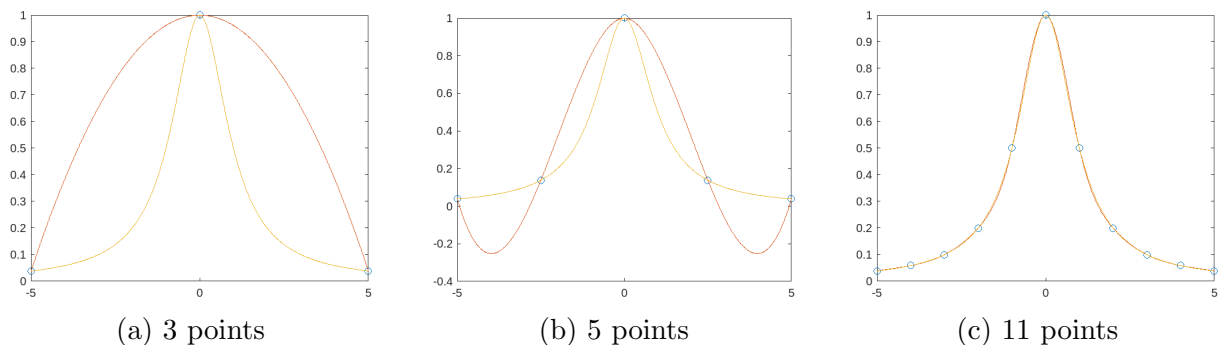


FIGURE 7 – Comparaison de l'interpolation de la fonction $f(x) = \frac{1}{(1+x^2)}$ sur $[-5, 5]$ pour 3, 5, et 11 points.

— Exemple 2 : Comparaison de l'interpolation de $f(x) = \cos(x)$ pour 3, 7 et 15 points.

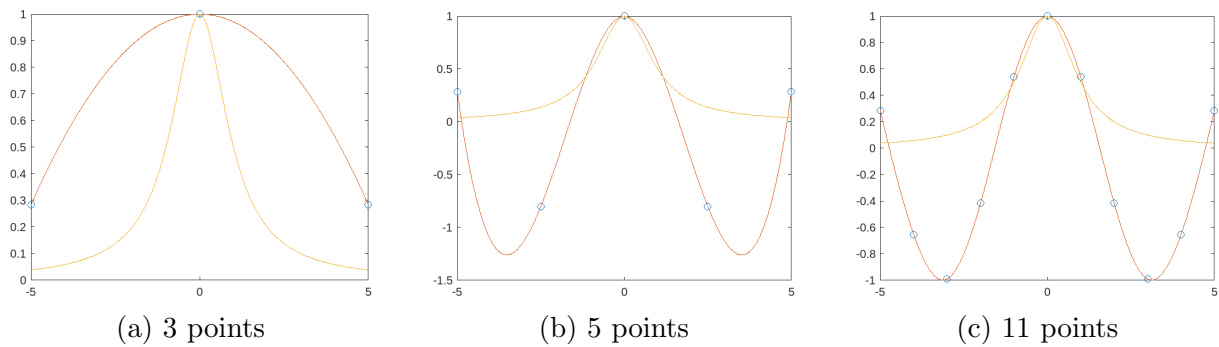


FIGURE 8 – Comparaison de l'interpolation de la fonction $f(x) = \cos(x)$ sur $[-5, 5]$ pour 3, 7 et 15 points.

5 Exemple d'application d'une étude numérique d'interpolation

Les méthodes d'interpolation ont souvent été utilisées, notamment lorsqu'on a des mesures à temps fixes, ce qui permet de déterminer la valeur prise par la fonction entre deux prises de mesure. Un exemple simple et concret où l'on utilise les méthodes d'interpolation étudiées dans ce TP est en météorologie. En effet, les stations météorologiques ne prennent pas des mesures en continu, elles les prennent à des intervalles réguliers (toutes les heures par exemple). On obtient donc des données sous forme de points discrets, comme la température relevée à 8h, 9h, 10h, etc.

Mais si l'on souhaite estimer la température à un instant entre deux mesures, par exemple à 8h30, on peut utiliser la méthode d'interpolation. Si on suppose que la température évolue de façon constante entre deux mesures (pas de front froid ou de pic de chaleur), la méthode la plus simple est l'interpolation linéaire, et elle s'écrit :

$$T(t) = T_i + \frac{T_{i+1} - T_i}{t_{i+1} - t_i}(t - t_i)$$

Cette approche fonctionne bien lorsque les variations sont douces. En revanche, si les températures changent rapidement, ce modèle ne convient plus puisque des oscillations risquent d'apparaître. Dans ce cas on peut alors utiliser une fonction spline cubique pour l'interpolation. Ainsi, la température pourra être modélisée avec une courbe plus lisse qui suit mieux les valeurs réelles.

Ainsi, les fonctions splines permettent d'obtenir des estimations plus réalistes et plus fiables et sans créer d'oscillations indésirées.

En résumé, ce genre de techniques est essentiel dès qu'on veut reconstruire une information continue à partir de données ponctuelles, que ce soit pour l'analyse, la visualisation ou même la simulation de phénomènes physiques.

6 Conclusion

Ce TP nous a permis d'aborder de manière concrète les différentes techniques d'interpolation numérique et surtout de mieux comprendre leurs avantages et leurs inconvénients. Notre objectif était qu'à partir d'un nombre restreint de points, d'arriver à reconstituer au mieux une fonction inconnue. C'est d'ailleurs un défi fréquemment rencontré en sciences et en ingénierie notamment lorsqu'on travaille à partir de données discrètes.

Nous avons commencé par les méthodes classiques d'interpolation polynomiale, comme par exemples avec celles de Lagrange et de Newton. Ces approches sont simple sur le plan théorique, puisqu'elles sont assez simple à programmer. Mais nous avons remarqué qu'elles peuvent devenir instables lorsqu'on utilise trop de points, en particulier s'ils sont équidistants, et si la fonction que l'on cherche à approximer varie très fortement. C'est ce que l'on appelle le **phénomène de Runge**.

Pour contourner ce problème, nous avons découvert les **points de Tchebychev**, qui permettent de mieux répartir les erreurs et de rendre l'interpolation plus fiable. Une interpolation plus fiable et avec moins d'oscillations notamment aux extrémités de l'intervalle. On observe alors une nette amélioration des résultats pour des fonctions qui varie beaucoup dans l'intervalle d'étude.

Enfin, pour éviter des oscillations indésirées nous nous sommes tournés vers une approche plus locale : l'**interpolation par morceaux**. Nous avons alors pu remarqué que peu importe la méthode choisie, que ce soit celle avec des segments de droites (interpolation affine) ou celle avec des courbes plus lisses comme les **splines cubiques**, cette méthode d'interpolation par morceaux permet de mieux suivre la forme réelle de la fonction sans introduire d'oscillations excessives.

En résumé, ce TP nous a donné une vision claire et pratique des différents outils d'interpolations que l'on a à notre disposition et que l'on peut utiliser en fonction des fonctions que l'on doit approcher. Nous avons d'ailleurs aussi vu qu'il est plus qu'essentiel de bien choisir la méthode en fonction du problème à résoudre. Sans quoi, nous pourrions nous retrouver avec des aberrations. Cela nous rappelle aussi que derrière chaque méthode mathématique, il y a un équilibre que l'on doit trouver pour assurer la précision, stabilité et simplicité de nos modèles numériques.

7 Bibliographie

Références

- [1] SCHEID, J.-C., *Méthodes numériques – Chapitre 3B : Interpolation polynomiale*,
https://scheid.perso.math.cnrs.fr/Enseignement/MN/CM_MN_ch3B.pdf
- [2] BURDEN, R.L., FAIRES, J.D., *Numerical Analysis*, Cengage Learning, 10^e édition.
- [3] QUARTERONI, A., SACCO, R., SALERI, F., *Numerical Mathematics*, Springer, 2007.
- [4] BIBMATH.NET, *Polynôme interpolateur de Lagrange*,
<https://www.bibmath.net/dico/index.php?action=affiche&quoi=./p/polynomelagrange.html>
- [5] BIBMATH.NET, *Différence divisée*,
<https://www.bibmath.net/dico/index.php?action=affiche&quoi=./d/differencedivisee.html>
- [6] WIKIPEDIA, *Phénomène de Runge*,
https://fr.wikipedia.org/wiki/Ph%C3%A9nom%C3%A8ne_de_Runge

8 Annexes

```
1  clc;
2  clear;
3  close all;
4  f1 = @(x) cos(x);
5  f2 = @(x) 1 ./ (1 + x.^2);
6  a = -5;
7  b = 5;
8
9  n_values = [3, 5, 11];
10 for idx=1:length(n_values)
11     x = linspace(a,b,n_values(idx));
12     v = f2(x);
13     xx = a:0.01:b;
14     vvv1 = interp1(x,v,xx,'linear');
15     plot(x,v,'o',xx,vvv1,'-',xx,f2(xx))
16     pause(2);
17 end
```

FIGURE 9 – Programme Matlab pour les tracés de la fonction d'interpolation "linear" des fonction 1 et 2

```
1  clc;
2  clear;
3  close all;
4  f1 = @(x) cos(x);
5  f2 = @(x) 1 ./ (1 + x.^2);
6  a = -5;
7  b = 5;
8
9  n_values = [3, 5, 11];
10 for idx=1:length(n_values)
11     x = linspace(a,b,n_values(idx));
12     v = f2(x);
13     xx = a:0.01:b;
14     vvv1 = interp1(x,v,xx,'spline');
15     plot(x,v,'o',xx,vvv1,'-',xx,f2(xx))
16     pause(2);
17 end
```

FIGURE 10 – Programme Matlab pour les tracés de la fonction d'interpolation "spline" des fonction 1 et 2

Construction pratique pour une interpolation avec une fonction spline : Dans le cas d'une spline naturelle, la construction revient à résoudre un système linéaire tri-diagonal — un type de système très bien conditionné et rapide à résoudre. Ce système contient $n - 1$ équations pour $n - 1$ inconnues, qui correspondent aux valeurs des dérivées secondes aux nœuds intermédiaires. Une fois ces dérivées connues, on peut reconstruire les polynômes cubiques sur chaque sous-intervalle.


```

1 % Interpolation par la méthode des différences divisées de Newton
2
3 clc;
4 clear;
5 close all;
6
7 % Définition des fonctions f1 et f2
8 f1 = @(x) cos(x);
9 f2 = @(x) 1 ./ (1 + x.^2);
10
11 % Bornes de l'intervalle
12 a = -5;
13 b = 5;
14
15 % Choix du nombre de points pour l'interpolation
16 n_values = [3, 5, 11];
17
18 % Évaluation et tracé pour chaque valeur de n
19 x_fine = linspace(a, b, 1000); % Points pour affichage des courbes réelles
20 figure;
21 for idx = 1:length(n_values)
22     n = n_values(idx);
23     %x_n = linspace(a, b, n); % Points d'interpolation
24     x_n = (a+b)/2 + (b-a)/2 * cos(((2*[0:n-1]+1)*pi)/(2*(n+1)))
25     y_f1 = f1(x_n);
26     y_f2 = f2(x_n);
27
28     % Calcul des coefficients de Newton
29     coeffs_f1 = newton_coeffs(x_n, y_f1)% donne les coeffs du pol inter. de la 1ere
30     coeffs_f2 = newton_coeffs(x_n, y_f2)
31
32     % Évaluation des polynômes interpolants
33     y_interp_f1 = newton_eval(coeffs_f1, x_n, x_fine);
34     y_interp_f2 = newton_eval(coeffs_f2, x_n, x_fine);
35

```

FIGURE 11 – Programme Matlab pour les tracés avec l'interpolation de Tchebychev -
Partie 1

```

36 % Tracé des résultats
37 subplot(2, length(n_values), idx);
38 plot(x_fine, f1(x_fine), 'b-', 'LineWidth', 1.5); hold on;
39 plot(x_fine, y_interp_f1, 'r--', 'LineWidth', 1.5);
40 plot(x_n, y_f1, 'ko', 'MarkerFaceColor', 'k');
41 title(['Interpolation de cos(x), n=' num2str(n)]);
42 legend('cos(x)', 'Interp.', 'Points', 'Location', 'Best');
43 grid on;
44
45 subplot(2, length(n_values), idx + length(n_values));
46 plot(x_fine, f2(x_fine), 'b-', 'LineWidth', 1.5); hold on;
47 plot(x_fine, y_interp_f2, 'r--', 'LineWidth', 1.5);
48 plot(x_n, y_f2, 'ko', 'MarkerFaceColor', 'k');
49 title(['Interpolation de 1/(1+x^2), n=' num2str(n)]);
50 legend('1/(1+x^2)', 'Interp.', 'Points', 'Location', 'Best');
51 grid on;
52 end
53
54 % Fonction pour calculer les coefficients des différences divisées de
55 % Newton (algo de newton)
56 function coeffs = newton_coeffs(x, y)
57     n = length(x);
58     coeffs = y;
59     for j = 2:n
60         for i = n:-1:j
61             coeffs(i) = (coeffs(i) - coeffs(i-1)) / (x(i) - x(i-j+1));
62         end
63     end
64 end
65
66 % Fonction pour évaluer le polynôme interpolant de Newton (algo de newton)
67 function y_interp = newton_eval(coeffs, x_nodes, x_eval)
68     n = length(x_nodes);
69     y_interp = coeffs(n) * ones(size(x_eval));
70     for k = n-1:-1:1
71         y_interp = y_interp .* (x_eval - x_nodes(k)) + coeffs(k);
72     end
73 end

```

FIGURE 12 – Programme Matlab pour les tracés avec l'interpolation de Tchebychev -
Partie 2